

DISEÑO DE SISTEMAS DE INFORMACIÓN

“METAMAPA”

JUSTIFICACIONES DE DISEÑO

Trabajo Práctico Anual Integrador
-2025-

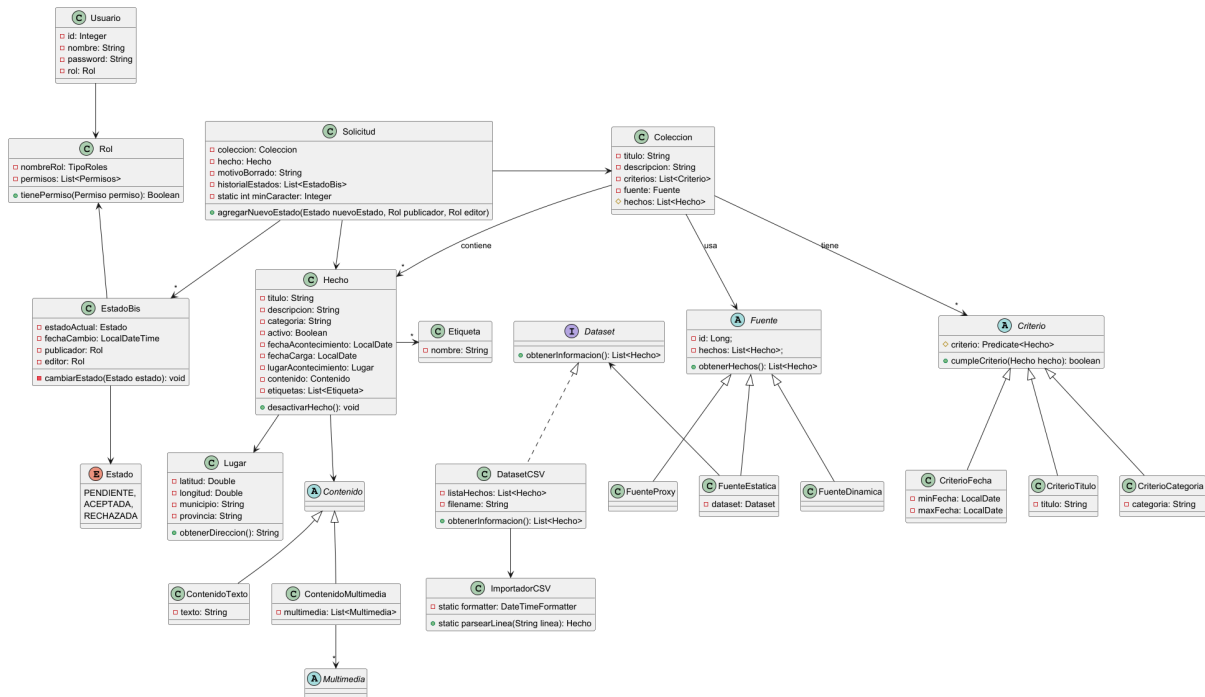
Grupo N°5



ENTREGA 2: Arquitectura y Modelado en Objetos - Parte II

Entregables

1. Modelo del Dominio: diagrama de clases inicial que contemple las funcionalidades requeridas.



2. Justificaciones de Diseño Iniciales.

Para los diferentes módulos se decidió llevar a cabo una estructura **multimódulo**. Este hace hincapié en la creación de módulos independientes para facilitar su modificación y escalabilidad. Así queda entonces el pom.xml del proyecto root (padre):

```
<modules>
  <module>fuenteDinamica</module>
  <module>fuenteEstatica</module>
  <module>fuenteProxy</module>
  <module>servicioAgregador</module>
</modules>
```

Se tomó esta decisión considerando las siguientes ventajas:

- Mejor organización y separación lógica: Cada módulo tiene su propio pom.xml, dependencias específicas y modelos.
- Spring hace posible un manejo superior de proyectos multimodulos.
- Reutilización de código: Es posible crear un módulo common para compartir DTOs, utilidades, enums, etc. sin duplicar código.
- Compilación e integración unificada: Se construye todo con un solo comando (mvn clean install).
- IDE friendly: IntelliJ permite trabajar con multimodulos, además de facilitar la navegación y refactorización.
- Facilita despliegue futuro como microservicios: Cada módulo ya está desacoplado, lo que ofrece escalabilidad.
- Aislamiento de potenciales errores: un error afecta la funcionalidad de un módulo en vez de afectar al programa completo.*

Teniendo en cuenta los **patrones de arquitectura de Software**, esta entrega se llevó a cabo bajo el **patrón de capas**. Los componentes se organizan en capas con tareas diferenciadas. Cada capa es un todo coherente (cohesión), con un rol único en el sistema. Las capas superiores usan servicios de las inferiores, pero no así de forma contraria o saltando niveles.

El acoplamiento unidireccional significa que la dependencia entre las diferentes capas del sistema va en una sola dirección.

En este caso:

- Los servicios conocen y dependen de los repositorios y las entidades de dominio.
- Los repositorios conocen y dependen de las entidades de dominio.
- Las entidades de dominio no conocen ni dependen de los servicios o repositorios.

Es por ello que se realizaron actualizaciones respecto a la entrega 1:

Se reestructuró el proyecto para aplicar el patrón de arquitectura MVC (Modelo Vista Controlador). El diagrama de clases se corrigió de forma tal que se respetara la unidireccionalidad, donde las entidades de dominio no conocen a los servicios y repositorios.

Esta estructura unidireccional es importante porque:

- Evita dependencias circulares.
- Mantiene las entidades de dominio independientes y reutilizables.
- Facilita el mantenimiento y los cambios en el sistema.

A continuación se explicará cada módulo y sus características.

FUENTE ESTÁTICA

El módulo de fuente estática permite la consulta de hechos a partir de Datasets. No es posible la modificación de aquellos hechos que provienen de fuentes estáticas. Los hechos no se persisten. Se realizó:

CAPA DE ENTIDADES:

Hecho: Entidad fundamental con sus atributos y métodos. Contiene la lógica de negocio pura: reglas, validaciones, entidades y objetos de valor. En este caso es el objeto que se obtiene a partir de la información del dataset (título; descripción; categoría; lugar del acontecimiento; fecha del acontecimiento).

Dataset: Contiene la lógica para lectura de un dataset. En esta entrega solo se cubre la lectura de archivos de tipo .csv.

CAPA DE SERVICIOS:

DatasetService: Se encarga de obtener una lista de Hechos a partir de un Dataset predefinido.

FUENTE PROXY

Se desea que las fuentes proxy sean capaces de consumir APIs externas. Serán consumidas en tiempo real y se espera que sus resultados sean siempre actuales. No es posible la modificación de aquellos hechos que provienen de APIs. Los hechos no se persisten. Se realizó:

CAPA DE ENTIDADES:

Hecho: Entidad fundamental con sus atributos y métodos. Contiene la lógica de negocio pura: reglas, validaciones, entidades y objetos de valor. En este caso modela la información obtenida de las diferentes APIs externas.

CAPA DE SERVICIOS:

ProxyService: Interfaz encargada de obtener y persistir los Hechos a partir de una API predefinida (MetaMapa, Disilab, etc.). Posee los métodos: obtener todos los hechos y obtener Hecho por Id.

MetaMapa Service: Se espera a través de WebClient poder comunicarse mediante protocolo HTTP con la API REST de MetaMapa.

Disilab Service: Se espera a través de WebClient poder comunicarse mediante protocolo HTTP con la API REST de Disilab. Se reciben hechos en formato JSON. Además se encarga de la autenticación con Bearer Token.

FUENTE DINÁMICA

Obtiene hechos a partir de las contribuciones de los usuarios, quienes pueden solicitar la modificación o eliminación de un hecho dinámico. Permite gestionar la subida de información en formato tanto de texto como multimedia. Los contribuyentes del Sistema pueden subir hechos en forma anónima o registrada. Se realizó:

CAPA DE ENTIDADES:

Hecho: Posee los atributos propios de un Hecho, además de la información del Contribuyente que lo subió. Contiene la lógica de negocio: reglas para su modificación y validaciones de permisos de usuarios.

Usuario: Modela los atributos del usuario registrado, como nombre de usuario y contraseña. Contiene la lógica para el manejo de roles y permisos.

CAPA DE REPOSITORIOS:

HechoRepository: Capa de persistencia de datos. Encapsula el acceso al medio persistente, que puede ser: memoria, archivos, bases de datos, etc. Por el momento es solo en memoria. Es posible buscar, agregar, modificar y eliminar hechos cargados por los usuarios.

UsuarioRepository: Por el momento es solo en memoria. Es posible buscar, agregar, modificar y eliminar usuarios persistidos. El objetivo es persistir la creación de cuentas por parte de usuarios registrados.

CAPA DE SERVICIOS:

HechoService: En este caso llama a HechoRepository para obtener los hechos necesarios o modificarlos. Controla que sucede con el Hecho según el tipo de usuario que lo sube.

FuenteDinamicaService: En este caso llama a HechoService para obtener los hechos necesarios o modificarlos. Gestiona y valida la carga de Hechos por parte de los Contribuyentes.

SERVICIO AGREGADOR

Permitirá que las personas visitantes y/o contribuyentes de la plataforma accedan a hechos de una colección provenientes de todas las fuentes. Por lo tanto, se permite ahora que las colecciones contengan hechos de distintas fuentes, sean estas estáticas, dinámicas o provenientes de servicios externos (fuentes proxy). Además, para posibilitar la futura exposición REST de las mismas, se incorporará un atributo textual identificador (handle) que será un string alfanumérico, sin espacios, único entre todas las colecciones.

CAPA DE ENTIDADES:

Colección: Entidad fundamental con sus atributos y métodos. Contiene la lógica de negocio: agrupa hechos bajo determinadas condiciones.

Solicitud: Contiene la lógica para el manejo de solicitudes por parte de usuarios.

CAPA DE REPOSITARIOS:

ColeccionRepository: Capa de persistencia de datos. Encapsula el acceso al medio persistente, que puede ser: memoria, archivos, bases de datos, etc. Por el momento es solo en memoria. Es posible buscar, editar, eliminar y crear colecciones. A su vez, se debe poder agregarles hechos.

SolicitudContribuyenteRepository: Capa de persistencias de datos. Se encarga de persistir las solicitudes que le llegan al administrador. Se separa en tres dependiendo del tipo de solicitud llegada:

- SolicitudEdicionContribuyenteRepository.
- SolicitudEliminacionContribuyenteRepository.
- SolicitudRevisionContribuyenteRepository.

CAPA DE SERVICIOS:

ColeccionService: Utiliza la lógica del dominio, repositorios y llama a APIs externas para obtener la información de valor que se necesita. En este caso llama a ColeccionRepository (para hacer CRUD de colecciones) y a los servicios de las Fuentes para obtener los hechos necesarios.

FuenteEstaticaService: A través de WebClient se comunica con la API REST que expone el módulo de Fuente Estática.

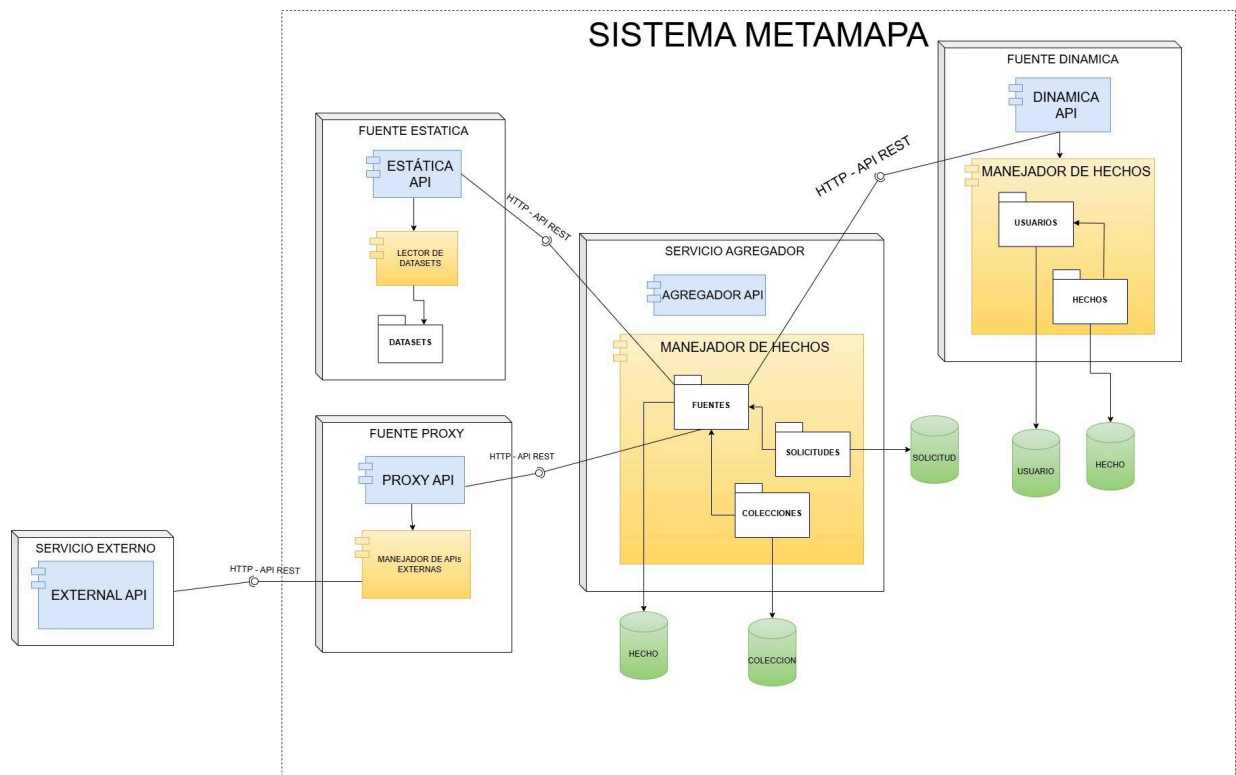
FuenteProxyService: A través de WebClient se comunica con la API REST que expone el módulo de Fuente Proxy.

FuenteDinamicaService: A través de WebClient se comunica con la API REST que expone el módulo de Fuente Dinámica.

SolicitudContribuyenteService: Usado para que un administrador pueda gestionar las solicitudes asignadas. Esta se divide en tres dependiendo del tipo de solicitud llegada:

- SolicitudEdicionContribuyenteService.
- SolicitudEliminacionContribuyenteService.
- SolicitudRevisionContribuyenteService.

3. Diagrama de Arquitectura.



4. Implementación de los requerimientos.

Link al repositorio: <https://github.com/dds-utn/2025-tpa-ma-ma-grupo-05/tree/entrega2>